

1. Analyse des Besoins

1.1. Acteurs du Système

L'application distingue trois types d'acteurs avec des niveaux d'accès différents :

Acteur	Rôle	Accès
Super Administrateur	Configure l'application, gère les utilisateurs et les types de demandes	Total
Administrateur	Traite les demandes, met à jour les statuts, ajoute des observations	Gestion des demandes
Agent OTC	Soumet et suit ses propres demandes administratives	Ses demandes uniquement

1.2. Fonctionnalités par Acteur

Super Administrateur

- Création, modification et suppression des comptes utilisateurs
- Gestion des rôles (Administrateur, Agent)
- Création et gestion des types de demandes avec champs dynamiques
- Activation ou désactivation des comptes utilisateurs
- Consultation des statistiques globales

Agent OTC

- Authentification et modification du mot de passe
- Soumission d'une nouvelle demande avec pièces jointes
- Suivi de l'état d'avancement en temps réel
- Consultation de l'historique des demandes
- Confirmation d'une demande acceptée
- Réception des notifications par email et SSE
- Tableau de bord personnel (statistiques)

Administrateur

- Consultation et filtrage de toutes les demandes reçues

- Prise en charge, validation ou rejet d'une demande
- Ajout d'observations et de commentaires
- Tableau de bord global avec statistiques et répartition par type

Cycle de Vie d'une Demande

Statut	Déclencheur	Acteur
PENDING	Soumission par l'agent	Agent
IN_PROGRESS	Prise en charge par l'admin	Admin
ACCEPTED	Validation par l'admin	Admin
REJECTED	Rejet par l'admin	Admin
CONFIRMED	Confirmation par l'agent	Agent

2. Architecture Technique

2.1. Vue d'Ensemble

Architecture client-serveur classique avec une séparation stricte entre le frontend et le backend. Les deux communiquent exclusivement via une API REST sécurisée par JWT. Pour les notifications en temps réel, le backend expose un endpoint SSE (Server-Sent Events) que le frontend maintient ouvert en permanence.

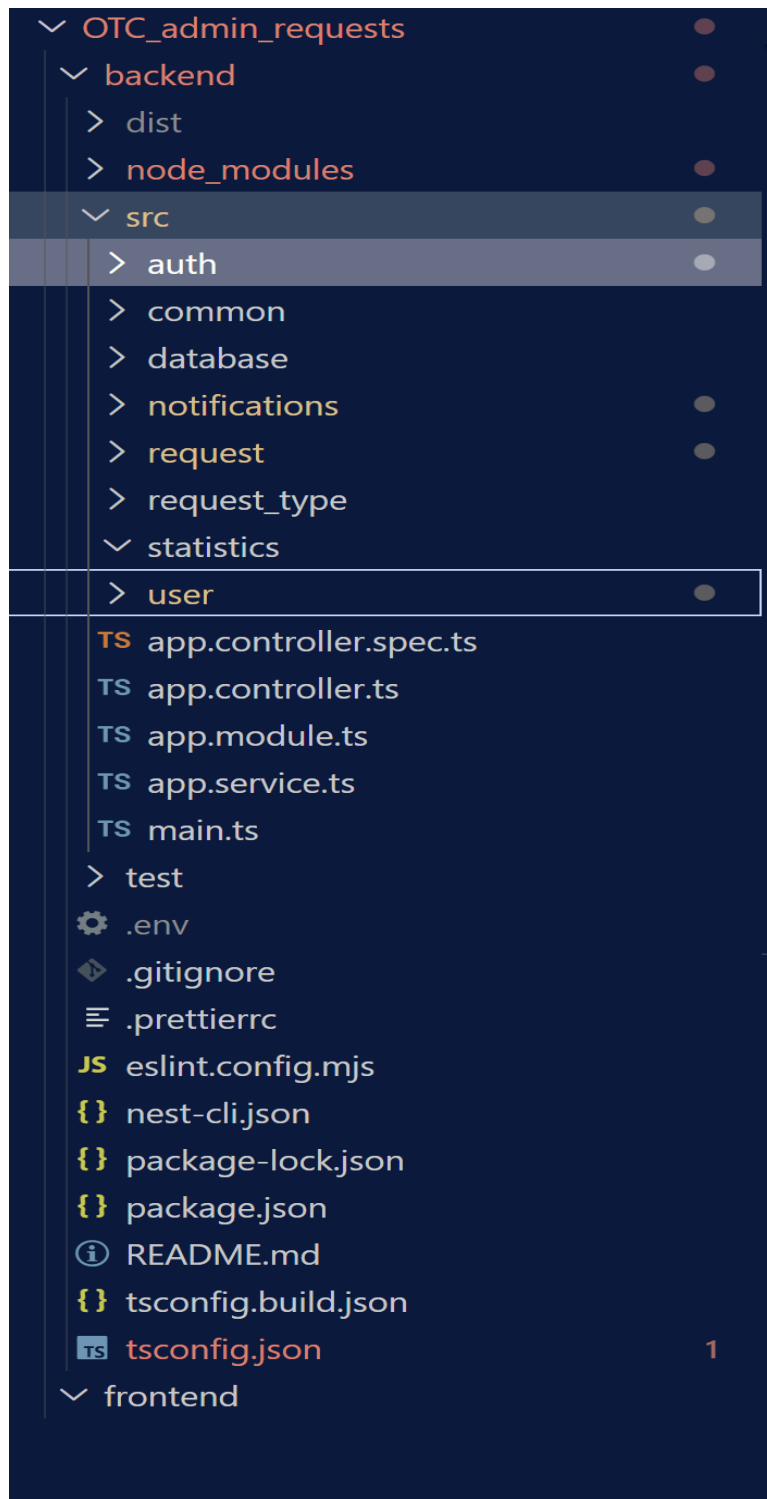
2.2. Stack Technologique

Couche	Technologie	Rôle
Backend	NestJS (Node.js / TypeScript)	API REST, logique métier, authentification
ORM	TypeORM	Mapping objet-relationnel, migrations
Base de données	PostgreSQL	Persistance des données
Authentification	JWT + Passport.js	Sécurisation des endpoints
Emails	Nodemailer + Mailtrap	Envoi de notifications par email
Temps réel	SSE (Server-Sent Events)	Notifications push vers le client
Frontend	React + TypeScript	Interface utilisateur
Documentation API	Swagger (OpenAPI)	Documentation interactive des endpoints

2.3. Architecture Backend: NestJS

Le backend est organisé en modules indépendants, chacun responsable d'un domaine fonctionnel précis.

Module	Responsabilité	Fichiers principaux
AuthModule	Authentification JWT, login, changement de mot de passe	auth.service.ts, jwt.strategy.ts, roles.guard.ts
UserModule	CRUD utilisateurs, gestion des rôles, activation	user.service.ts, user.controller.ts
RequestTypeModule	Types de demandes dynamiques avec champs configurables	request_type.service.ts, request_type_field.entity.ts
RequestModule	Soumission, suivi, filtrage et gestion des demandes	request.service.ts, request.controller.ts
NotificationModule	Emails automatiques et notifications SSE temps réel	mail.service.ts, sse.service.ts, notification.listener.ts



2.4 Architecture Frontend: React

Le frontend sera développé avec React et TypeScript. Chaque acteur (Agent, Admin, Super Admin) accède à une interface dédiée protégée par des guards de navigation.

- Pages Agent : tableau de bord, soumettre une demande, historique, profil
- Pages Admin : liste des demandes, filtrage, détail, mise à jour de statut, statistiques
- Pages Super Admin : gestion des utilisateurs, gestion des types de demandes

- Composants partagés : navbar, notifications SSE, badges de statut

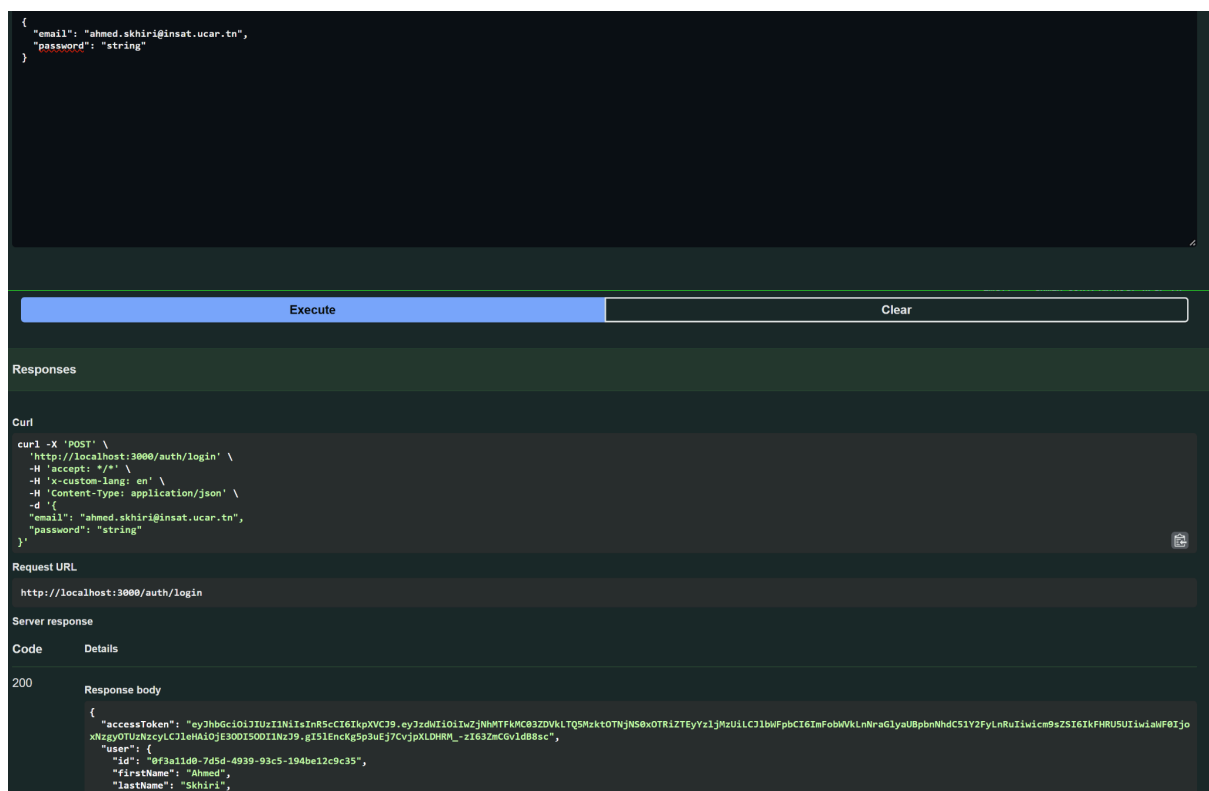
3. Implémentation Backend

Chaque module NestJS est composé d'un contrôleur (gestion des routes HTTP), d'un service (logique métier), d'entités TypeORM (mapping base de données) et de DTOs (validation des données entrantes).

3.1. Authentification et Autorisation

L'authentification est implémentée avec JWT (JSON Web Token) et Passport.js. Le flux d'authentification est le suivant :

- L'agent envoie ses identifiants (email + mot de passe) à POST /auth/login
- Le serveur vérifie les credentials avec bcrypt et génère un token JWT signé
- Le token est envoyé dans chaque requête via le header Authorization: Bearer <token>
- JwtStrategy valide le token et attache l'utilisateur complet à la requête
- RolesGuard vérifie que le rôle de l'utilisateur correspond au rôle requis par le endpoint



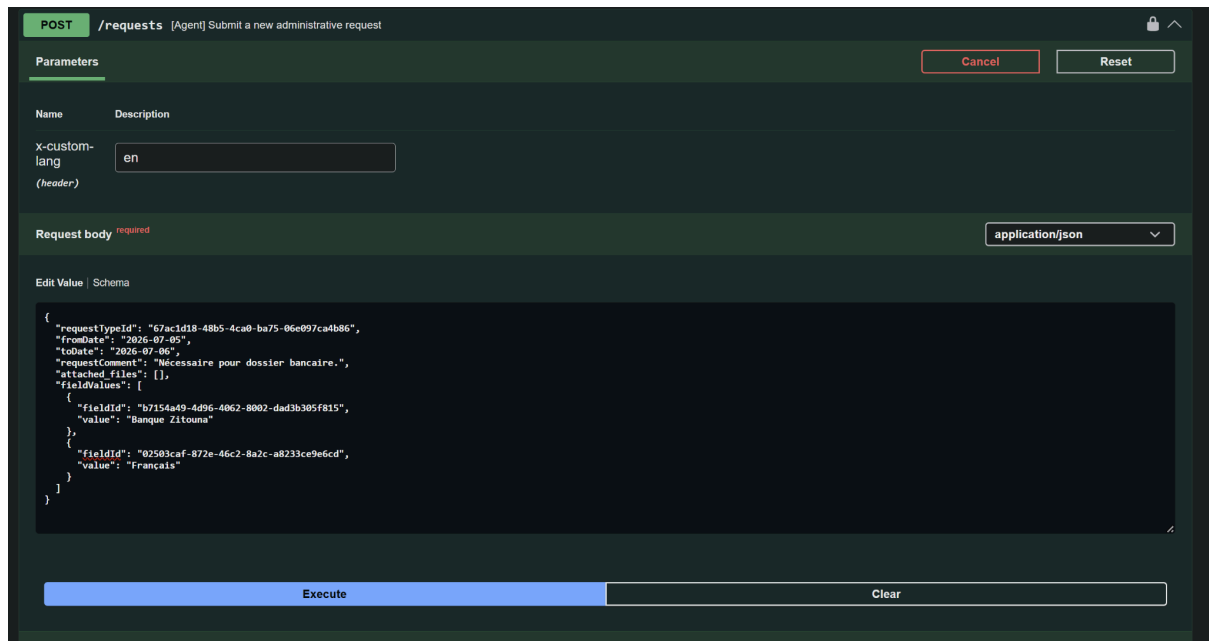
3.2. Gestion des Demandes

Le RequestService implémente la logique complète du cycle de vie d'une demande :

- Validation des dates

- Validation des champs dynamiques : vérification que tous les champs requis sont renseignés
- Pagination et filtrage des listes (par type, agent, statut, période)
- Statistiques agrégées par statut pour les tableaux de bord

L'agent envoie une demande



POST /requests [Agent] Submit a new administrative request

Parameters Cancel Reset

Name	Description
x-custom-lang	en

(header)

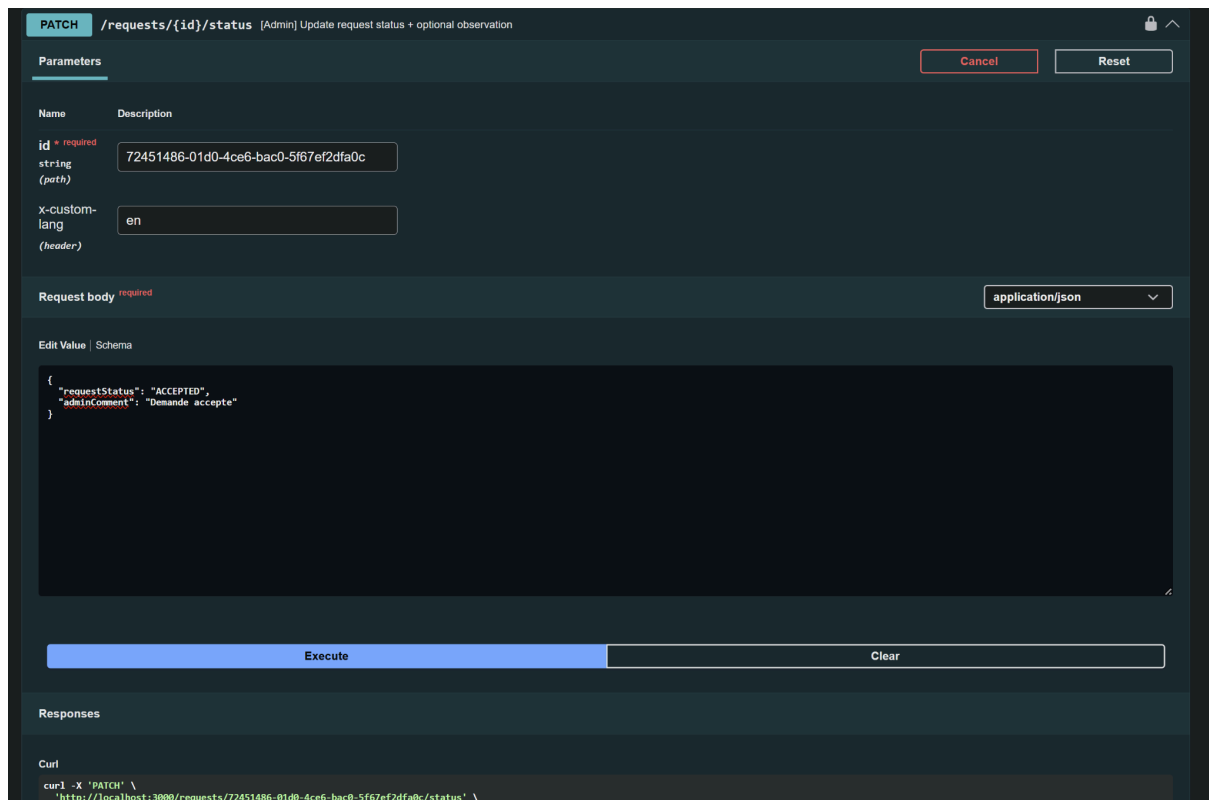
Request body required application/json

Edit Value | Schema

```
{
  "requestId": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86",
  "fromDate": "2026-07-05",
  "toDate": "2026-07-06",
  "requestComment": "Nécessaire pour dossier bancaire.",
  "attachedFiles": [],
  "fieldValues": [
    {
      "fieldId": "b7154a49-4d96-4062-8002-dad3b305f815",
      "value": "Banque Zitouna"
    },
    {
      "fieldId": "02503caf-872e-46c2-8a2c-a8233ce9e6cd",
      "value": "Français"
    }
  ]
}
```

Execute Clear

Puis un Admin reçoit la demande et modifie le statut



PATCH /requests/{id}/status [Admin] Update request status + optional observation

Parameters Cancel Reset

Name	Description
id required	72451486-01d0-4ce6-bac0-5f67ef2dfa0c

string (path)

Name	Description
x-custom-lang	en

(header)

Request body required application/json

Edit Value | Schema

```
{
  "requestStatus": "ACCEPTED",
  "adminComment": "Demande acceptée"
}
```

Execute Clear

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:3000/requests/72451486-01d0-4ce6-bac0-5f67ef2dfa0c/status' \
  -H 'Content-Type: application/json'
```

3.3. Types de Demandes Dynamiques

Le super-administrateur peut créer des types de demandes entièrement configurables. Les types disponibles dans l'application sont :

Type de demande	Champs principaux
Congé annuel	Durée, Date de reprise, Motif (optionnel)
Attestation de travail	Destinataire, Langue
Attestation de salaire	Destinataire, Langue, Période
Demande de matériel	Type de matériel, Quantité, Justification
Ordre de mission	Destination, Date départ, Date retour, Objet, Budget
Autres demandes	Objet de la demande, Pièces jointes requises

3.4 Système de Notifications

Le système de notifications combine deux mécanismes complémentaires :

Event Emitter

Le service utilise EventEmitter2 de NestJS pour émettre des événements métier de manière découplée. Trois événements sont définis :

- request.created : émis à la soumission d'une demande
- request.status_updated : émis à chaque changement de statut
- request.confirmed : émis quand l'agent confirme une demande acceptée

Server-Sent Events (SSE)

Le endpoint GET /notifications/stream maintient une connexion persistante HTTP avec chaque utilisateur connecté. Quand un événement survient, le serveur pousse instantanément une notification au client concerné sans que celui-ci ait besoin de faire une nouvelle requête.

Emails automatiques

Nodemailer est utilisé pour envoyer des emails HTML formatés. Les scénarios de notification sont :

Événement	Destinataire(s)	Canal
Nouvelle demande soumise	Agent (confirmation) + Tous les admins	Email + SSE
Statut mis à jour	Agent concerné	Email + SSE
Demande confirmée par l'agent	Agent + Tous les admins	Email + SSE

Après l'envoi de la demande

Demande #7 soumise



From: OTC Admin <689649f0479e8a>
To: <ahmed.skhiri@insat.ucar.tn>
[Show Headers](#)

2026-07-02 00:59, 575 Bytes

Looks good? Send it to real users
Go live with Email API/SMTP - 4,000 production emails included

Try API/SMTP

HTMLHTML SourceTextRawSpam AnalysisHTML CheckTech Info



Bonjour **Ahmed Skhiri**,

Votre demande **Attestation de travail (#7)** a bien été soumise et est en attente de traitement.

Vous serez notifié dès qu'elle sera traitée.

Après le traitement de la demande

Demande #7 – statut mis à jour : ACCEPTED



From: OTC Admin <689649f0479e8a>
To: <ahmed.skhiri@insat.ucar.tn>
[Show Headers](#)

2026-07-02 01:03, 670 Bytes

Looks good? Send it to real users
Go live with Email API/SMTP - 4,000 production emails included

Try API/SMTP

HTMLHTML SourceTextRawSpam AnalysisHTML CheckTech Info



Bonjour **Ahmed Skhiri**,

Le statut de votre demande **Attestation de travail (#7)** a été mis à jour.

Nouveau statut : ACCEPTED

Observation : Demande **accepte**

API REST : Documentation

L'API est documentée avec Swagger (OpenAPI) et accessible à l'adresse /api pendant le développement. Voici un résumé des endpoints principaux :

Méthode	Endpoint	Rôle requis	Description
POST	/auth/login	Public	Connexion — retourne un token JWT
POST	/auth/change-password	Tous	Changement de mot de passe
GET	/request-types	Tous	Lister les types de demandes actifs
POST	/request-types	Super Admin	Créer un type avec champs dynamiques
POST	/requests	Agent	Soumettre une nouvelle demande
GET	/requests/my	Agent	Historique des demandes de l'agent
GET	/requests/my/stats	Agent	Statistiques personnelles
PATCH	/requests/:id/confirm	Agent	Confirmer une demande acceptée
GET	/requests	Admin	Toutes les demandes avec filtres
PATCH	/requests/:id/status	Admin	Mettre à jour le statut
GET	/requests/stats	Admin	Statistiques globales
GET	/notifications/stream	Tous	Connexion SSE temps réel
POST	/users	Super Admin	Créer un utilisateur
PATCH	/users/:id/toggle-active	Super Admin	Activer/désactiver un compte

 **Swagger**
Powered by SMARTBEAR



API 1.0 OAS 3.0

API docs

Authorize 

Notifications



GET	/notifications/stream	[Any] SSE stream		
-----	-----------------------	------------------	---	---

Users



POST	/users	[SuperAdmin] Create a new user		
GET	/users	[SuperAdmin] Get all users, optionally filtered by role		
GET	/users/{value}	[SuperAdmin] Find user by id, name, email, or phone		
PATCH	/users/{id}	[SuperAdmin] Update user fields		
DELETE	/users/{id}	[SuperAdmin] Delete a user		
PATCH	/users/{id}/toggle-active	[SuperAdmin] Activate or deactivate a user		
PATCH	/users/{id}/reset-password	[SuperAdmin] Reset a user password		

Requests



POST	/requests	[Agent] Submit a new administrative request		
GET	/requests	[Admin] Get all requests filter by type, agent, date, status		
GET	/requests/my	[Agent] Get own request history (paginated + filtered)		
GET	/requests/my/stats	[Agent] Get personal dashboard statistics		
GET	/requests/my/{id}	[Agent] Get one of own requests by id		
POST	/requests/{id}/comments	[Agent / Admin] Append a comment to an open request		
POST	/requests/{id}/attachments	[Agent] Attach files to an open request		
PATCH	/requests/{id}/confirm	[Agent] Confirm an accepted request		
GET	/requests/stats	[Admin] Get global dashboard statistics		
GET	/requests/{id}	[Admin] Get any request by id		
DELETE	/requests/{id}	[Super-Admin] Hard-delete a request record		
PATCH	/requests/{id}/status	[Admin] Update request status + optional observation		

Request Types			^
POST	/request-types	[SuperAdmin] Create a request type with its fields	🔒 ↓
GET	/request-types	[All] Get all active request types	🔒 ↓
GET	/request-types/admin/all	[SuperAdmin] Get all request types including inactive	🔒 ↓
GET	/request-types/{id}	[All] Get one request type with its fields	🔒 ↓
PATCH	/request-types/{id}	[SuperAdmin] Update request type metadata	🔒 ↓
DELETE	/request-types/{id}	[SuperAdmin] Delete a request type	🔒 ↓
POST	/request-types/{id}/fields	[SuperAdmin] Add fields to an existing request type	🔒 ↓
DELETE	/request-types/fields/{fieldId}	[SuperAdmin] Remove a field from a request type	🔒 ↓
Auth			^
POST	/auth/login	Login	↓
POST	/auth/change-password	[Any authenticated user] Change own password	🔒 ↓

Exemples de reponses:

Envoi demande:

Code		Details
201	Response body	
	<pre>{ "id": "72451486-01d0-4ce6-bac0-5f67ef2dfa0c", "requestNumber": 7, "user": { "id": "0f3a11d0-7d5d-4939-93c5-194be12c9c35", "firstName": "Ahmed", "lastName": "Skhiri", "email": "ahmed.skhiri@insat.ucar.tn", "password": "\$2b\$10\$hJm/wekMUQ8q/XwqgbyhuJzLSoJyxx1DJPTaelTRfTrBZmMlaQa2", "role": "AGENT", "isActive": true, "totalRequests": 0, "createdAt": "2026-07-01T09:04:16.102Z", "updatedAt": "2026-07-01T09:04:16.102Z", "phoneNumber": "94949494", "department": "nice department" }, "userId": "0f3a11d0-7d5d-4939-93c5-194be12c9c35", "requestType": { "id": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86", "name": "Attestation de travail", "description": "Demande d'une attestation de travail.", "isActive": true, "createdAt": "2026-07-01T10:01:30.041Z", "fields": [{ "id": "b7154a49-4d96-4062-8002-dad3b305f815", "name": "Date de la demande", "type": "date", "required": true }] } }</pre>	
	Response headers	

```

Response body
  userId : 073a11d0-7d5d-4939-93c5-1940e12c9c35 ,
  "requestType": {
    "id": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86",
    "name": "Attestation de travail",
    "description": "Demande d'une attestation de travail.",
    "isActive": true,
    "createdAt": "2026-07-01T10:01:30.041Z",
    "fields": [
      {
        "id": "b7154a49-4d96-4062-8002-dad3b305f815",
        "fieldName": "Destinataire",
        "fieldType": "text",
        "isRequired": true,
        "description": "Organisme ou administration destinataire",
        "requestTypeId": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86"
      },
      {
        "id": "02503caf-872e-46c2-8a2c-a8233ce9e6cd",
        "fieldName": "Langue",
        "fieldType": "text",
        "isRequired": true,
        "description": "Français ou Arabe",
        "requestTypeId": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86"
      }
    ]
  },
  "requestTypeId": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86",
  "requestStatus": "PENDING",

```

Login :

Code

Details

Response body

```
{  
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiJlZGVhdHRkNC02YmNkLTRlODAtNmRzMGM3NGVxZWQwMnRlVmIlLCJlbWFrpbCjE6mhbbWRpLmhmbiwgcG90eS0biisjaXBGUOIjBRElTlTiSmldClGTMTyKINDEyMCwiZXswImFjdGdyOTgweTIwQi.c1t29acciaQMAY4zmgy37DpdRJioMRd3BeITh-RlsIo",  
  "user": {  
    "id": "bdeai4dd-6bcd-4b80-adc1-74f1c142debb",  
    "firstName": "string",  
    "lastName": "string",  
    "email": "hamdi.hamdi@otc.tn",  
    "role": "ADMIN",  
    "isActive": true,  
    "totalRequests": 0,  
    "createdAt": "2026-06-29T18:01:12.405Z",  
    "updatedAt": "2026-06-29T18:01:12.405Z",  
    "phoneNumber": "123456789999",  
    "department": "string"  
  }  
}
```

Download

Modification du statut de la demande:

200

Response body

```
]
},
"requestTypeId": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86",
"requestStatus": "ACCEPTED",
"requestDate": "2026-07-02T00:59:05.683Z",
"requestComment": "Nécessaire pour dossier bancaire.",
"attached_files": [],
"comments": [
  "[ADMIN|bdea14d4-6bcd-4b80-adc1-74f1c142debb|2026-07-02T01:03:11.482Z] Demande acceptée"
],
"fromDate": "2026-07-05T00:00:00.000Z",
"toDate": "2026-07-06T00:00:00.000Z",
"fieldValues": [
  {
    "id": "87c11103-04a3-4e76-874c-150e7d8632c7",
    "value": "Banque Zitouna",
    "requestId": "72451486-01d0-4ce6-bac0-5f67ef2dfa0c",
    "field": {
      "id": "b7154a49-4d96-4062-8002-dad3b305f815",
      "fieldName": "Destinataire",
      "fieldType": "text",
      "isRequired": true,
      "description": "Organisme ou administration destinataire",
      "requestTypeId": "67ac1d18-48b5-4ca0-ba75-06e097ca4b86"
    },
    "fieldId": "b7154a49-4d96-4062-8002-dad3b305f815"
  },
]
}
```

Création d'un compte agent(Super Admin)

201

Response body

```
{
  "id": "b836877b-2ab8-492e-b07c-2188b9c723eb",
  "firstName": "string",
  "lastName": "string",
  "email": "omar.ahmed@otc.tn",
  "password": "$2b$10$LbvWS23WtEJJTX6RyaT4M0hNwUSnNoWzZEguyHY4QQEXujOFcuR0a",
  "role": "AGENT",
  "isActive": true,
  "totalRequests": 0,
  "createdAt": "2026-07-02T01:16:19.282Z",
  "updatedAt": "2026-07-02T01:16:19.282Z",
  "phoneNumber": "1234444555",
  "department": "string"
}
```

Tous les comptes utilisateurs(super admin)

200

Response body

```
[
  {
    "id": "fc75b8e1-db1e-40ea-b2d5-2613be760072",
    "firstName": "string",
    "lastName": "string",
    "email": "mohamed.ahmed@otc.tn",
    "password": "$2b$10$om3/DMtj7jbi00YGBX/r6eEEhzLQtu8CNIFQONbqECDtxdU55sNMK",
    "role": "AGENT",
    "isActive": true,
    "totalRequests": 0,
    "createdAt": "2026-06-25T10:03:20.619Z",
    "updatedAt": "2026-06-25T10:03:20.619Z",
    "phonenumber": "string",
    "department": "string"
  },
  {
    "id": "3cd6c584-0965-4bfc-bee3-45bca945537c",
    "firstName": "string",
    "lastName": "string",
    "email": "ahmed.ahmed@otc.tn",
    "password": "$2b$10$i1/96a3jDAxL50QnSB000FNtskGwGfqX04H67BCoXdmHiZtqVanK",
    "role": "AGENT",
    "isActive": true,
    "totalRequests": 0,
    "createdAt": "2026-06-29T09:25:31.357Z",
    "updatedAt": "2026-06-29T09:34:33.512Z",
    "phonenumber": "1234566",
    "department": "string"
  }
]
```

Création d'un nouveau type de demande(congé, attestation...)(Super Admin)

201

Response body

```
{
  "id": "2792b9e8-59aa-4383-8ce1-53a22c4e0749",
  "name": "Un bouveau type",
  "description": "Rune description",
  "isActive": true,
  "createdAt": "2026-07-02T01:18:41.802Z",
  "fields": [
    {
      "id": "06fcb762-7060-4711-bf44-d7c1fd30b017",
      "fieldName": "destination",
      "fieldType": "text",
      "isRequired": true,
      "description": "une deescription du champ",
      "requestTypeId": "2792b9e8-59aa-4383-8ce1-53a22c4e0749"
    }
  ]
}
```

Frontend

Pour la partie frontend, les interfaces de l'application seront développées avec React
Voici quelques exemples de dashboards pour chaque utilisateur suivant son rôle

Dashboard super admin

User Management

Filter

Role

Status

Ajouter un utilisateur

Nom	Prenom	Email	Numero	Departement	Role	Activite	Nombre demandes	created at	updated at	Actions

Request type management

type

ajouter un type

Type	Description	Date de creation	Active	Date de modification	Nombre total de demandes	Demandes en cours	Demandes accepte	Demandes rejete		Actions

Dashboard agent

search filter by number

type

status

Date

new request

request managment table

Pending	En cours de traitement	accepted/rejected
request card		

Je peux également utiliser des templates React comme <https://mui.com/material-ui/getting-started/templates/crud-dashboard/> qui seront ensuite adaptés et personnalisés selon la charte graphique de l'OTC